Competitive Programming Notes

Pakin Olanraktham

March 21, 2026

Contents

1 Data Structures and Algorithms **2**

1.1 Data Structures 2

1.1.1 Wavelet Tree 2

1.2 Dynamic Programming Optimization 4

1.2.1 Convex Hull Trick 4

1.2.2 Li-Chao Tree 6

1.2.3 Divide and Conquer 8

1.3 Tree Algorithms 9

1.3.1 Centroid Decomposition 9

1.4 Miscellaneous 11

1.4.1 Subset Sum in $\mathcal{O}(N\log(N) + \frac{N\sqrt{N}}{64})$ 11

1.4.2 Inverse Mod in $\mathcal{O}(n)$ 12

2 Problems **13**

2.1 APIO 13

2.1.1 APIO 2023 13

2.2 China Team Homework 15

2.2.1 IOI 2026 China National Training Team Homework 15

Chapter 1

Data Structures and Algorithms

1.1 Data Structures

1.1.1 Wavelet Tree

Wavelet Tree is a Data Structure mainly for finding the k'th minimum value in some range $[l, r]$. The tree is built dynamically, meaning that we will use pointers.

```
struct Wavelet {
    vector <int> p;
    Wavelet *l, *r;
};
```

When we build the tree, we check if the b'th bit is 0 or 1. If it is 0, we put that element into the left side. Otherwise, we put in the right side. The vector `p` is a prefix sum array counting number of elements in the left side.

```
void build(const vector <int> &A, int b) {
    if (b == 0 || A.empty()) return;
    l = new Wavelet();
    r = new Wavelet();
    int c = 0;
    vector <int> L, R;
    for (int x : A) {
        if (x & (1 << (b-1))) R.emplace_back(x);
        else L.emplace_back(x), c++;
        p.emplace_back(c);
    }
    l->build(L, b-1);
    r->build(R, b-1);
}
```

When we query, we count the number of elements in the left side in range l to r. If the number of elements in the left side is more than or equal to k, we recursive to the right side. Otherwise we go to the right side. We have to change the value of l, r, k too

when moving down to the right side.

```
int qr(int l, int r, int k, int b) { // [l, r), k, 0-index
    if (b == 0 || p.empty()) return 0;
    int pr = (r > 0 ? p[r-1] : 0);
    int pl = (l > 0 ? p[l-1] : 0);
    int cnt = pr - pl;
    if (k < cnt) return this->l->qr(pl, pr, k, b-1);
    else return (1 << (b-1)) + this->r->qr(l-pl, r-pr, k-cnt, b-1);
}
```

Note that these functions are inside the `Wavelet` struct.

1.2 Dynamic Programming Optimization

1.2.1 Convex Hull Trick

Convex Hull Trick is an algorithm to find $\min_{j<i}(f_j(x))$ or $\max_{j<i}(f_j(x))$ efficiently.

First, we convert the function $f(x)$ to $mx + c$, which x depends only on i, m and c depends only on j. For example

```
dp[i] = max(dp[j] + x[i]*y[i] - x[j]*y[i] - a[i])
      = max((-x[j])*y[i] + dp[j]) + x[i]*y[i] - a[i]
```

As we can see, m = -x[j], x = y[i], c = dp[j]. Note that if we want to do CHT, the slope **must** be monotonic. Otherwise, we have to use Li-Chao Tree.

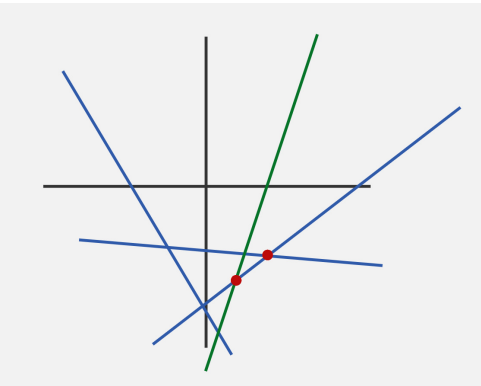
Now, how do we compute the maximum or minimum? We know that the recurrence is a line with monotonic slope.

Inserting

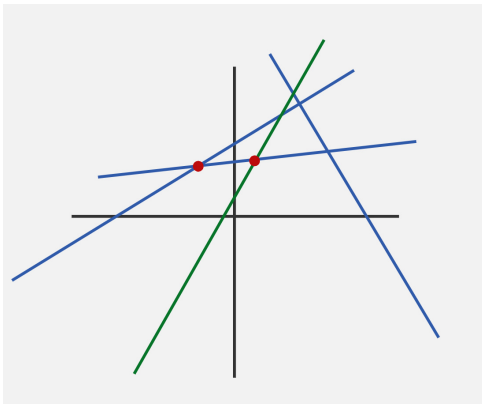
We will use `deque <int>` to store the lines indices.

For computing maximum, we want to maintain the lines as lower hull shaped. For minimum, we want the lines as upper hull shaped.

Before inserting, we have to check whether the old line is useless or not. We can do that by checking the intersection point between the [new line, top] and [top, top-1]. (You should try drawing on paper by yourself.)



Lower Hull (Maximum)



Upper Hull (Minimum)

For finding the intersection point we can do

```
double m(int x, int y) {
    return (double) (c[x] - c[y]) / (m[y] - m[x]);
}
```

Querying

Monotone Query

If the hull is maintained correctly for each type of query, we will always query from front for non-decreasing queries, and always from back for non-increasing queries.

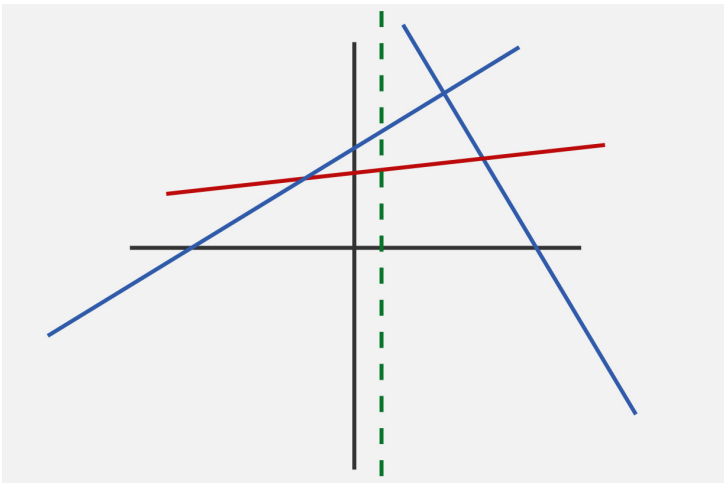
We will check the top and top-1. For maximum, if $f_{\text{top}}(x) \leq f_{\text{top-1}}(x)$, we pop the top. For minimum, if $f_{\text{top}}(x) \geq f_{\text{top-1}}(x)$, we pop the top. For example

```
while (sz > 1 && calc(cht[0], p[i]) <= calc(cht[1], p[i])) {
    cht.pop_front(), sz--;
}
```

Then the answer will be $f_{\text{top}}(x) + c$ where c is a constant that depends only on x.

Arbitrary Query

We perform a binary search for the line that contain x.



```
int l = 0, r = sz-2, idx = sz-1;
while (l <= r) {
    int mid = l + (r-l)/2;
    if (m(cht[mid], cht[mid+1]) <= x) l = mid+1;
    else r = mid-1, idx = mid;
}
```

1.2.2 Li-Chao Tree

Reference : A Simple Introduction to Li-Chao Segment Tree

Li-Chao Tree is an upgrade of Convex Hull Trick. Instead of monotone slope in CHT, it can handle both arbitrary slope and query.

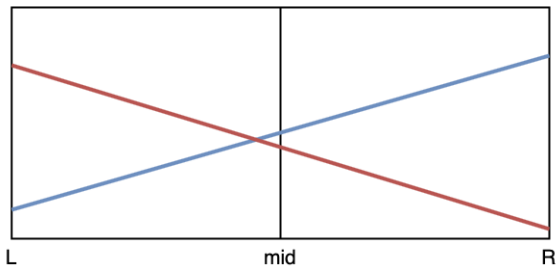
Given 2 types of queries, add a line to multiset S, find the maximum value at some point x. It can handle both of the query with $\mathcal{O}(\log X)$ where X is the maximum value of type-2 query. We will use a Segment Tree to store the best line for some range $[l, r]$. We can even do lazy propagation, sparse (dynamic node creation), or persistent with it.

We will use each node of the segment tree as a Line.

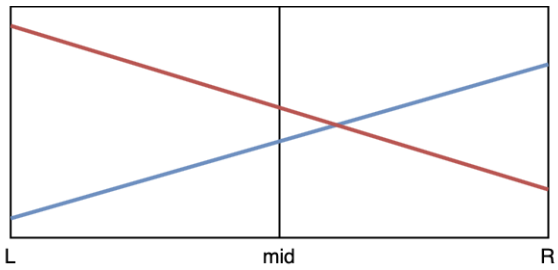
```
struct Line {
    ll m = 0, c = LLONG_MIN; // c = LLONG_MAX for min Li-Chao Tree
    ll operator()(ll x) { return m * x + c; }
} seg[4*mxX];
```

Insertion

For conveinence, we assume $m_{\text{new}} < m_{\text{old}}$ (if not, then swap the lines). We will consider 2 cases.



Case 1



Case 2

Case 1 – $f_{\text{new}}(\text{mid}) < f_{\text{old}}(\text{mid})$: The old line is better for range $[l, r]$, except some range in the left side, so we recursively insert the new line to the left.

Case 2 – $f_{\text{new}}(\text{mid}) > f_{\text{old}}(\text{mid})$: The new line is better for range $[l, r]$, except some range in the right side, so we recursively insert the old line to the right.

```
void ins(int l, int r, Line x, int u) {
    if (l == r) {
        if (seg[u](l) < x(l)) seg[u] = x;
        return;
    }
    int m = l + (r-l)/2;
    if (x.m > seg[u].m) swap(x, seg[u]);
    if (x(m) < seg[u](m)) ins(l, m, x, u<<1);
    else swap(x, seg[u]), ins(m+1, r, x, u<<1|1);
}
```

Query

We just query for point x normally, but we have to get the maximum with the current line too.

```
ll qr(int l, int r, int x, int u) {
    if (l == r) return seg[u](x);
    else {
        int m = l + (r-l)/2;
        if (m >= x) return max(seg[u](x), qr(l, m, x, u<<1));
        else return max(seg[u](x), qr(m+1, r, x, u<<1|1));
    }
}
```

1.2.3 Divide and Conquer

Divide and Conquer is also a technique we can use to optimize dynamic programming.

Consider this DP formula

$$\text{dp}[k][i] = \min_{j \le i}(\text{dp}[k-1][j-1] + C(j,i))$$

such that we can calculate $C(i,j)$ in $\mathcal{O}(1)$. If we solve this normally, it would take $\mathcal{O}(KN^2)$.

We can use Divide and Conquer technique to optimize this DP formula. Let $\text{opt}(k,i)$ be the optimal value of j that minimize the value of the RHS of the equation. This technique will **only work** if $\text{opt}(k,i-1) \le \text{opt}(k,i)$ for all i, which is hard to prove.

From Quadrangle Inequality, if we found out that $C(a,c)+C(b,d) \le C(a,d)+C(b,c)$ satisfies for all $a < b < c < d$, then $\text{opt}(k,i-1) \le \text{opt}(k,i)$ holds (the proof of this has been left as an exercise to the reader).

Since calculating $\text{dp}[k][i]$ doesn't need other value in $\text{dp}[k]$, so we can calculate each i independently. This is the code for the algorithm.

```
void dnc(int l, int r, int optl, int optr) { // [l, r] is the
    current range, [optl, optr] is the range we need to check.
    if (l > r) return;
    int m = l + (r-l)/2, opt;
    dp[k][m] = INT_MAX;
    for (int i = optl; i <= optr; i++) {
        if (dp[k][m] > dp[k-1][i-1] + C(i, m)) {
            dp[k][m] = dp[k-1][i-1] + C(i, m);
            opt = i;
        }
    }
    dnc(l, m-1, optl, opt); // for all i < m, opt(k, i) <= opt(k, m)
    dnc(m+1, r, opt, optr); // for all i > m, opt(k, i) >= opt(k, m)
}
```

1.3 Tree Algorithms

1.3.1 Centroid Decomposition

Centroid

A centroid of a tree is defined as a node such that if removed, no connected component has more than $\frac{N}{2}$ nodes.

We can find a centroid in a tree by selecting an arbitrary node in the tree. Then loop through its children's subtree. If the size of a child's subtree is more than $\frac{N}{2}$, traverse down that node. Otherwise, if no child's subtree has size more than $\frac{N}{2}$, then that node is the centroid.

```
void fsz(int u, int p) {
    sz[u] = 1;
    for (auto v : adj[u]) if (v != p) fsz(v, u), sz[u] += sz[v];
}

int ct(int u, int p) {
    for (auto v : adj[u]) if (sz[v] * 2 > n) return ct(v, u);
    return u;
}
```

Properties

1. If the centroid of a tree isn't unique, there are exactly two centroids which are adjacent. Removing the edge between them divides the tree into two connected components of the equal sizes.
2. When a leaf is added to or removed from a tree, the centroid changes by at most one node.
3. When two trees are connected, the centroid of the resulting tree lies on the path between the centroids of the original trees.

Centroid Decomposition

Centroid Decomposition is a divide-and-conquer technique for trees, which recursively splits the tree by removing its centroid. The resulting decomposition tree has a height of at most $\mathcal{O}(\log N)$.

Properties

1. The height of the decomposed tree is at most $\log N$
2. Every path between u and v can be decomposed into two parts: u to $\text{lca}_{ctd}(u, v)$ and $\text{lca}_{ctd}(u, v)$ to v. We can use this property for querying and updating. In the **Xania and Tree** problem, we update and query by traversing up to the $\log N$ parents for each node.

```
int fsz(int u, int p) {
    sz[u] = 1;
    for (auto v : adj[u]) if (v != p && !vis[v]) sz[u] += fsz(v, u);
    return sz[u];
}

int ct(int u, int p, int tsz) {
    for (auto v : adj[u]) if (v != p && !vis[v]) if (sz[v] * 2 > tsz
        ) return ct(v, u, tsz);
    return u;
}

void ctd(int u, int p) {
    int C = ct(u, u, fsz(u, u));
    vis[C] = 1;
    if (p == -1) p = C;
    par[C] = p;
    for (auto v : adj[C]) if (!vis[v]) ctd(v, C);
}
```

1.4 Miscellaneous

1.4.1 Subset Sum in $\mathcal{O}(N\log(N) + \frac{N\sqrt{N}}{64})$

Problem 1.4.1. *Given an array $a_1, a_2, a_3, \ldots, a_n$ where $\sum_{i=1}^{n} a_i \leq n$. Find all possible subset sum.*

If we did this naively, it would take $\mathcal{O}(N^2)$, but we can use some trick to reduce it to $\mathcal{O}(N\log(N) + \frac{N\sqrt{N}}{64})$.

Lemma 1.4.2. *Let N be a positive integer. We can form every integer from 1 to N as a sum by using each element from the multiset $\{2^0, 2^1, 2^2, \ldots, 2^k, r\}$ at most once, where $\sum_{i=0}^{k} 2^i \leq N < \sum_{i=0}^{k+1} 2^i, r = N - \sum_{i=0}^{k} 2^i$.*

Proof. Using $2^0, 2^1, 2^2, \ldots, 2^k$, we can form a sum of every number from 1 to $\sum_{i=0}^{k} 2^i$. Next, if we want to form a sum $N \geq x > \sum_{i=0}^{k} 2^i$, we can use r and some combination that has the sum of $x - r = x - N + \sum_{i=0}^{k} 2^i \leq \sum_{i=0}^{k} 2^i$. □

Now, we can continuously decompose the number. Let `cnt`$[x]$ be the number of times that integer x appears in the array. We will decompose into $S = \{x{\cdot}2^0, x{\cdot}2^1, \ldots, x{\cdot}2^k, x{\cdot}r\}$, then increase `cnt`$[e], e \in S$. Each number takes at most $\mathcal{O}(\log N)$ time, so this operation is $\mathcal{O}(N\log N)$ in total.

Since $\sum_{i=1}^{n} a_i \leq n$, we know that there can be at most about $\sqrt{N}$ different numbers. So, the total time complexity will be $\mathcal{O}(N\sqrt{N})$ computing the subset sum naively, which with `std::bitset` will reduce the time to $\mathcal{O}(\frac{N\sqrt{N}}{64})$.

So, we can solve the problem in $\mathcal{O}(N\log(N) + \frac{N\sqrt{N}}{64})$.

1.4.2 Inverse Mod in $\mathcal{O}(n)$

Normally, finding inverse mod of x takes $\mathcal{O}(\log m)$ time. But what if $x \leq n$? Yes, we can precompute in $\mathcal{O}(n)$ and answer in $\mathcal{O}(1)$.

$$
\begin{aligned}
m &\equiv q \cdot i + r && \pmod{m} \\
0 &\equiv \lfloor m/i \rfloor \cdot i + (m \bmod i) && \pmod{m} \\
-(m \bmod i) &\equiv \lfloor m/i \rfloor \cdot i && \pmod{m} \\
i^{-1} &\equiv -\lfloor m/i \rfloor \cdot (m \bmod i)^{-1} && \pmod{m} \\
i^{-1} &\equiv m - \lfloor m/i \rfloor \cdot (m \bmod i)^{-1} && \pmod{m}
\end{aligned}
$$

```
inv[1] = 1;
for (int i = 2; i <= n; i++) {
    inv[i] = (mod - (mod / i) * inv[mod % i] % mod) % mod;
}
```

Chapter 2

Problems

2.1 APIO

2.1.1 APIO 2023

APIO23_cyberland - Cyberland

Date : 03/11/2026
Score : 97/100
Tags : `Shortest Path, Dijkstra`

Statement Given a weighted undirected graph of n nodes and m edges, $n, m \leq 10^5$. Each node has an ability of one of three types:

1. **Type 0** : Reset the current cost to zero.
2. **Type 1** : Do nothing.
3. **Type 2** : Divide the total distance by 2.

Find the minimum cost from node 0 to node H. You can perform the "Type 2" operation at most $K \leq 10^6$ times.

Subtask 1–7 : $K \leq 30$ (97 Points)
Subtask 8 : $K \leq 10^6$ (3 Points)

My Observation This problem can be transformed into a "multi-source shortest path" problem where the sources are node 0 and any nodes with ability Type 0. We can then perform DP on Dijkstra. However, this state-space approach ($N \times K$) only works for small values of K.

Editorial Moving from the 97-point solution to the 100-point solution involves a precision observation. We can prove that using $K = \min(K, 70)$ is sufficient to make the absolute error less than 10^{-7}.

- Let the optimal solution use the Type 2 ability V times, where $V \gg 70$.
- Consider only the last 70 usages of the ability. Let p be the first node that uses the ability in this truncated path.
- The shortest path from 0 to p is at most $N \times \max(C)$.
- By ignoring earlier divisions, the added cost is at most $\frac{N \times \max(C)}{2^{70}} \approx 0$.

Code : https://github.com/PakinDioxide/temp/tree/main/APIO/APIO23_cyberland

APIO23_sequence - Sequence

Date : 03/11/2026
Score : 70/100
Tags : `Persistent Segment Tree, Two Pointers`

Statement Given an array a size of N. Let $f(l, r, x)$ be the number of occurrence of x in range (l, r). For some range (l, r), let the medians are $\lfloor \frac{l+r}{2} \rfloor, \lceil \frac{l+r}{2} \rceil$ indices after the array in that range is sorted. Find the maximum value of $f(l, r, x)$ when x is one of the medians of the array in range (l, r).

My Observation I did a binary search to find the maximum value, x. Then, iterate over a to find that is it possible for value $a[i]$ to be the median, which appear at least x time. We can do this by finding the left most index j that number $a[i]$ appear x times in the range (i, j). Then we want to find that is it possible that $a[i]$ will be the median.

1. Let $fx = \sum_{k=i}^{j} b[k]$, where $b[k] = \max(-1, \min(1, a[k] - a[i]))$.
2. Let $MX = \max_{k=0}^{i-1} \left(\sum_{l=k}^{i-1} c[l]\right) + \max_{k=j+1}^{N-1} \left(\sum_{l=j+1}^{k} c[l]\right)$, where $c[k] = (a[k] \geq a[i] \to 1, a[k] < a[i] \to -1)$
3. Let $MN = \min_{k=0}^{i-1} \left(\sum_{l=k}^{i-1} d[l]\right) + \max_{k=j+1}^{N-1} \left(\sum_{l=j+1}^{k} d[l]\right)$, where $d[k] = (a[k] > a[i] \to 1, a[k] \leq a[i] \to -1)$
4. Then $MX = MX + fx, MN = MN + fx$.
5. So, $a[i]$ will be able to be the median if $(mn, mx) \cap (-x, x) \neq \emptyset$.

This solution is $\mathcal{O}(n \log^2(n))$, which got 70 points. Note that in step 1, 2, and 3, we can use persistent segment tree to handle these queries.

Editorial Transforming $\mathcal{O}(n \log^2(n))$ to $\mathcal{O}(n \log(n))$ is very easy. We just do a "two-pointers" over the same value instead of binary searching.

Code : https://github.com/PakinDioxide/temp/tree/main/APIO/APIO23_sequence

2.2 China Team Homework

2.2.1 IOI 2026 China National Training Team Homework

QOJ833 Cells Blocking

Date : 03/10/2026
Tags : `Observation`

Statement Given a $n \times m$ grid, each is either a free cell or a blocked cell. Find number of ways to choose 2 different cells which block all path from $(0, 0)$ to $(n-1, m-1)$.

Editorial We know that all paths must be between the "left-most" path and the "right-most" path.

- **Case 1: We only have to block one cell**
  If the left-most path intersects with the right-most path, it means all paths must pass through that cell. In this case, we can block just that cell and any other arbitrary cell.
- **Case 2: We need to block two cells**
  Now, we will block a cell along the "left-most" path. Then find the new "left-most" path. We can do this by moving to the next cell in the same $i + j$ diagonal. Then, count the number of intersections (excluding the cells from "Case 1").

Code : https://github.com/PakinDioxide/temp/tree/main/CNHW/2026/1.cpp

QOJ837 Giant Penguin

Date : 03/13/2026
Tags : `Centroid Decomposition, DFS Tree`

Statement Given an undirected graph with n nodes and m edges, $n \leq 10^5, m \leq 2 \times 10^5$. Each node is on at most $k \leq 10$ cycles. You will perform $q \leq 2 \times 10^5$ operations.

1. Mark a node u.
2. Find the shortest distance from any marked node to node u.

Editorial Consider this problem, when the graph is a tree, how would you do it. The obvious solution would be doing a centroid decomposition, like the *Xenia and Tree* problem. But because of this problem is a grpah, we can't do it.

Consider a centroid node c, there will be at most k edges connecting different subtrees (since c can have at most k cycles). We can just do a BFS from one of the end of that

edge, and find the shortest path from it to the others. We call that node *special*.

When we do query Type 1, we can traverse up its centroid parents, and also each parent's *special* node. We will update by $\text{ans}[c][sp] = \min(\text{ans}[c][sp], \text{dist}[c][sp][u])$. For Type 2, we can do similarly, $\text{ANS} = \min(\text{ANS}, \text{ans}[c][sp] + \text{dist}[c][sp][u])$.

But since $\text{dist}[c][sp][u]$ size is $\mathcal{O}(n^2k)$, we have to change c to $\text{dep}[c]$. This works because there is only one c that has depth $= \text{dep}[c]$ and is the parent of u.

Code : https://github.com/PakinDioxide/temp/tree/main/CNHW/2026/2.cpp

QOJ856 Cactus

Date : 03/13/2026
Tags : `Math, Combinatorics, DFS Tree`

Statement Given a cactus (a graph which each node is on atmost 1 cycle) with n nodes and m edges, $n \leq 3 \times 10^5, m \leq 4 \times 10^5$. Find the number of ways to paint this cactus by using one of the $k \leq 10^9$ colors, such that no two adjacent nodes have the same color.

Editorial Lets consider a simpler problem first.

1. **A path of n nodes** : $\text{path}[n] = k \times (k-1)^{n-1}$. Obviously.
2. **A cycle of n nodes** : $\text{cycle}[n] = \text{path}[n] - \text{cycle}[n-1]$. We know that the number of ways is the $\text{path}[n]$ minus the number of ways that the start and the end have the same color. We can convert this by merging the first and last node together, which will be the number of ways to color a cycle of $n-1$ nodes, i.e. $\text{cycle}[n-1]$.

Now, how about a cactus? We just multiply the current answer by $\frac{k-1}{k}$ for each edge connecting different group. This works because each color is symmetric. So we just want to block a color out of k colors.

Code : https://github.com/PakinDioxide/temp/tree/main/CNHW/2026/4.cpp

QOJ857 Social Distancing

Date : 03/21/2026
Tags : `Observation, DFS`

Statement Given a tree with $n \leq 2000$ nodes. You have two sets S_1 and S_2 which each element are independent (no two elements are connected by an edge). You have to find whether you can transform S_1 into S_2 by moving an element to another adjacent node such that all element are always independent with less than $4 \cdot n^2$ operations. If you can, print the operations too (you do not need to minimize the operation).

Editorial The first thing you have to observe is that if both S_1 and S_2 can transform into S, then S_1 can be transform into S_2. So what S should we choose? First, we will re-index the tree sorted by its depth, tie breaking randomly. Then, we want to find the minimum lexicographical order of S_1 and S_2. We can do this by checking if we can move a node to i from 1 to n. If yes, we move it. After considering i, we will not touch i anymore. This works because we consider i from the lowest depth. But before checking if there exist a node that can move to i, we have to try to push all node away from i first. If after these operations, S_1 and S_2 are the same, then the answer is yes, else no.

Code : https://github.com/PakinDioxide/temp/tree/main/CNHW/2026/5.cpp

QOJ862 Social Justice

Date : 03/14/2026
Tags : `Binary Search`

Statement Given a set $\{A_1, A_2, A_3, \ldots, A_n\}$. Let a subset $S \subset A$ be valid if $\max(S) \leq K \times \frac{\sum_i S}{\mathrm{n}(S)}$. Let $x = \max(\mathrm{n}(S_{\text{valid}}))$. Find the set $\{A_1, A_2, A_3, \ldots, A_n\} - (T_1 \cup T_2 \cup \cdots \cup T_n)$, where T is the set of all valid S which $\mathrm{n}(S) = x$.

Editorial First, we will sort A. Then, we can find x by doing a binary serach. We can check if $x = y$ is valid by doing a sliding window of size y. Finally, we can find if A_i could be in a valid S. We will go through all valid S which has no gap, then we will find the left most index which can replace S_1. We can do this by binary searching the index. This works because if $\{z, S_2, S_3, \ldots, S_n\}$ works, then $\{w, S_2, S_3, \ldots, S_n\}, w \geq z$ would work, because the maximum is the same, but the average is increased.

Code : https://github.com/PakinDioxide/temp/tree/main/CNHW/2026/6.cpp

QOJ962 Thanks to MikeMirzayanov

Date : 03/15/2026
Tags : `Divide and Conquer`

Statement Let an array A size of $n \leq 20,000$ be a permutaion of $\{1, 2, 3, \ldots, n\}$. In each operation, you will split the array into $k < n$ segments, $S_1, S_2, S_3, \ldots, S_k$. Then you reverse these segments, $S_k, S_{k-1}, S_{k-2}, \ldots, S_1$ (keeping the order of S_i the same). You have to perform this operation at most 120 times to make this array sorted.

Editorial

1. **What if the array consist of only 0 and 1 only?**
   First, we can merge the connecting number, like 0010111001 to 010101. Then, we can sort this array using $\log_3 n$ times. We can do this by pushing each 1 to the middle 1, like 010101 to 001110.
2. **Now, how do we move from 0 and 1 to our array A?**
   For each element, we can let that element be 0 if it is less than mid, and 1 if it is more then mid. Then we can just do this recursively. This will take $\frac{1}{2}\log_2 n \log_3 n$ operations.

Code : https://github.com/PakinDioxide/temp/tree/main/CNHW/2026/9.cpp

QOJ970 Best Subsequence

Date : 03/18/2026
Tags : `Persistent Segment Tree`

Statement Given an array A of length $n \leq 10^5$. There are $Q \leq 10^5$ queries. In each query, you have to find the minimum value of W such that there exist a subsequence $B_{1\ldots k}$ of $A_{l\ldots r}$ and $B_i + B_{i \bmod k} \leq W$.

Editorial

1. **We can binary search for W instead.**
   We know that if W works, then $W+1$ will definetly work. Also, for each W, if we can find a subsequence B such that n$(B) >= K$, then W will work.
2. **Now, how do we find the maximum size of B for each W?**
   First, we know that we should include all $A_i \leq \frac{W}{2}$, we call these element *good*. Then, between two *good* elements, we can add at most one more element, because these element between is more than $\frac{W}{2}$, so no two of these number can stay together. Also, the number we choose between two *good* elements should be the minimum. Then we can check if adding that number is possible.
3. **How do we find the maximum size of B for each W efficiently?**
   Let A_L be the right most index to the left that $A_L \leq A_i$ and R is the left most index to the right that $A_r \leq A_i$. We know that we can add A_i if $2 \times A_i \leq W$ or $\max(A_L + A_i, A_R + A_i) \leq W$. So we can use Persistent Segment Tree to count these number that is less than or equal to W between l and r efficiently. However, there might be some number that L or R is out of range, also the array B is circular, so we will find the left most index L_W and right most index R_W such that $2 \times A_{L_W} \leq W$ and $2 \times A_{R_W} \leq W$. Then, we will find the minimum between $(l, L_W - 1) \cup (R_W + 1, r)$, and check if adding that element is possible.

Total complexity : $\mathcal{O}(n \log_2^2(n))$

Code : https://github.com/PakinDioxide/temp/tree/main/CNHW/2026/11.cpp

QOJ975 Game

Date : 03/19/2026
Tags : `Convex Hull`

Statement Given an array A of length $n \leq 5 \times 10^5$. You start at some random point $k \in \{1, 2, 3, \ldots, n\}$. In each operation, you can choose to **stop** and receive A_i points, or **move** randomly to left or right with equal probability, i.e. $k-1, k+1$ each with 50% chance. Note that you can move only if you are not at 1 or n. Find the maximum expected value of points you can get.

Editorial Consider this, for each index i, you will stop when ever you reach index L or index R. The expected value of reaching L before R is $A_L \times \frac{R-i}{R-L}$, and $A_R \times \frac{i-L}{R-L}$ for R. This is exactly the y value of the line that goes through point (L, A_L) and (R, A_R) at point $x = i$. Now, we want to find the maximum expected value, which is exactly the definition of the Upper Convex Hull. So we can solve this problem in $\mathcal{O}(n)$.

Code : https://github.com/PakinDioxide/temp/tree/main/CNHW/2026/13.cpp